CODEMITVISHAL · FREE COURSE

Python Pro

From Python Foundations to **professional** Python: clean code, data structures, OOP, testing, packaging, concurrency and real projects.

13 modules

∞ runnable examples

0 € always free

github.com/CodeMitVishal/python-pro

COURSE MAP

From foundations to pro, in 13 modules

#	MODULE	YOU'LL BE ABLE TO…
08	**Pythonic code**	write short, idiomatic Python
09	**Data structures deep dive**	pick the right structure (Big-O)
10	**Advanced functions**	write closures and decorators
11	**Iterators & generators**	process data lazily
12	**Advanced OOP**	design clean classes (SOLID)
13	**Errors, context managers, logging**	fail safely, log clearly
14	**Project structure & packaging**	ship an installable CLI

#	MODULE	YOU'LL BE ABLE TO…
15	**Type hints & code quality**	catch bugs before running
16	**Testing with pytest**	prove your code works
17	**Concurrency**	use threads, processes, asyncio
18	**Data, APIs & databases**	build a REST API with a DB
19	**Performance & internals**	profile and speed things up
20	**Capstone project**	build it all, end to end

Every module: slides · runnable code · exercise + solution · quiz

github.com/CodeMitVishal/python-pro

MODULE

08

Pythonic code

Same results, half the code. Write Python the way experienced Python developers do.

IN THIS MODULE

Comprehensions	01_comprehensions.py
Unpacking	02_unpacking.py
Looping like a pro	03_looping_like_a_pro.py
Truthiness & the walrus :=	04_truthiness_and_walrus.py
Strings like a pro	05_strings_like_a_pro.py
Demo: grade report	06_demo_grade_report.py
Exercise: refactor to Pythonic	ex08_refactor_to_pythonic.py

LESSON · COMPREHENSIONS

Build a list in one readable line

```
[ p * 1.19 for p in prices if p < 20 ]
```

① what to keep ② the loop ③ optional filter

```
# the old way: 4 lines
with_tax = []
for p in prices:
    with_tax.append(p * 1.19)

# Pythonic: 1 line
with_tax = [p * 1.19 for p in prices]
```

KIND	SYNTAX
list	`[x for x in xs]`
dict	`{k: v for k, v in pairs}`
set	`{x.lower() for x in xs}`
generator	`sum(x for x in xs)`

> **if/else before for** chooses a value. **if after for** filters.

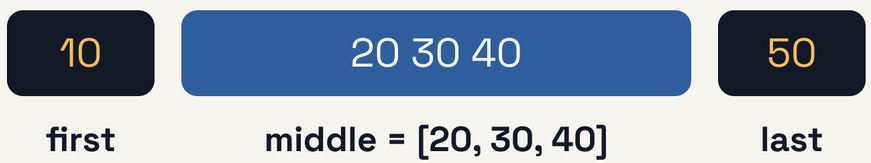
LESSON · UNPACKING

Pull values out and into names, in one step

```
x, y = (3, 7)                  # basic
a, b = b, a                    # swap, no temp
first, *middle, last = nums    # * = "the rest"
name, _, city = record         # _ = ignore

area(*[4, 5])                  # list -> args
area(**{"width": 2, "height": 9})

merged = [*odds, *evens]
settings = defaults | user     # later keys win
```



> The number of names must match the number of values, **unless** one name has a `*`.

> `*` spreads a list into arguments, `**` spreads a dict into keyword arguments.

LESSON · LOOPING LIKE A PRO

Stop writing range(len(...))

INSTEAD OF…	WRITE
`for i in range(len(xs)): xs[i]`	`for i, x in enumerate(xs, start=1)`
Two lists with one index	`for a, b in zip(xs, ys)`
A custom sort function	`sorted(xs, key=len, reverse=True)`
A flag + loop + break	`any(x < 0 for x in xs)`
”Is every item OK?” loop	`all(x >= 50 for x in xs)`
Track the biggest by hand	`max(pairs, key=lambda p: p[1])`
`for k in d: d[k]`	`for k, v in d.items()`

```
for s in scores:
    if s < 50:
        print("fail found")
        break
else:
    print("all passed")
```

> **for … else:** the `else` runs only if the loop finished **without** `break`.

> `zip` stops at the shortest list. Add `strict=True` to catch length bugs.

LESSON · TRUTHINESS & WALRUS

Short, clear conditions

FALSY

`0 0.0 "" [] {} set() None False`

Everything else is truthy, even `"0"` and `[0]`.

INSTEAD OF…	WRITE
`if len(cart) == 0:`	`if not cart:`
`if x == None:`	`if x is None:`
`x >= 50 and x < 80`	`50 <= x < 80`
if/else to pick a default	`name or "Guest"`

```
# walrus := assigns AND returns
while (line := input()) != "quit":
    print(line.upper())

if (n := len(data)) > 3:
    print(f"long list: {n} items")
```

⚠️ `count or 10` turns a real **0** into 10, because 0 is falsy. Use `10 if count is None else count`.

LESSON · STRINGS LIKE A PRO

f-strings format, align and even debug

SPEC	EXAMPLE	RESULT
`.2f`	`f"{1234.5678:.2f}"`	**1234.57**
`,.2f`	`f"{1234.5:,.2f}"`	**1,234.50**
`.1%`	`f"{0.4567:.1%}"`	**45.7%**
`05d`	`f"{42:05d}"`	**00042**
`<10 / >10`	`f"{name:<10}|"`	**left / right align**
`=`	`f"{user=}"`	**user='Vishal'**

```
" ".join(["learn", "python"])
# 'learn python'

name, age, city = line.split(",")
# split + unpack in one go
```

> Strings are **immutable**: every method returns a **new** string.

> Build big text with a list and `"".join()`, not `+=` in a loop.

DEMO · GRADE REPORT

Everything from Module 8 in one program

```
for line in raw.splitlines():
    name, *marks = line.split(",")          # unpacking
    students[name] = [int(m) for m in marks]  # list comp

averages = {n: sum(m) / len(m)                 # dict comp
            for n, m in students.items()}
ranking = sorted(averages.items(),
                 key=lambda p: p[1], reverse=True)

for place, (name, avg) in enumerate(ranking, 1):
    result = "PASS" if avg >= 50 else "FAIL"
    print(f"{place:<3}{name:<10}{avg:>6.1f}  {result}")

print(all(a >= 50 for a in averages.values()))
```

OUTPUT

```
#  Name          Avg  Result
1  Vishal       91.3  PASS
2  Li           85.3  PASS
3  Raj          70.7  PASS
4  Sam          45.0  FAIL

Everyone passed?  False
```

> Run it: `python 08_pythonic_code/06_demo_grade_report.py`

EXERCISE 8 · YOUR TURN

Refactor 9 functions to Pythonic code

The task

> Each function works, but it's written the long way. Rewrite each one with the **Module 8 tool that fits**. Keep the names and results the same.

> `money()` also has a bug: `1234.5` must become **€1,234.50**.

> Run the file: all built-in checks must pass.

comprehension · enumerate · zip · any · max(key=) · or · join

```
# before
result = []
for n in numbers:
    if n % 2 == 0:
        result.append(n * n)

# after
[n * n for n in numbers if n % 2 == 0]
```

Starter:
exercises/ex08_refactor_to_pythonic.py
Solution:
solutions/ex08_refactor_to_pythonic.py

MODULE

09

Data structures deep dive

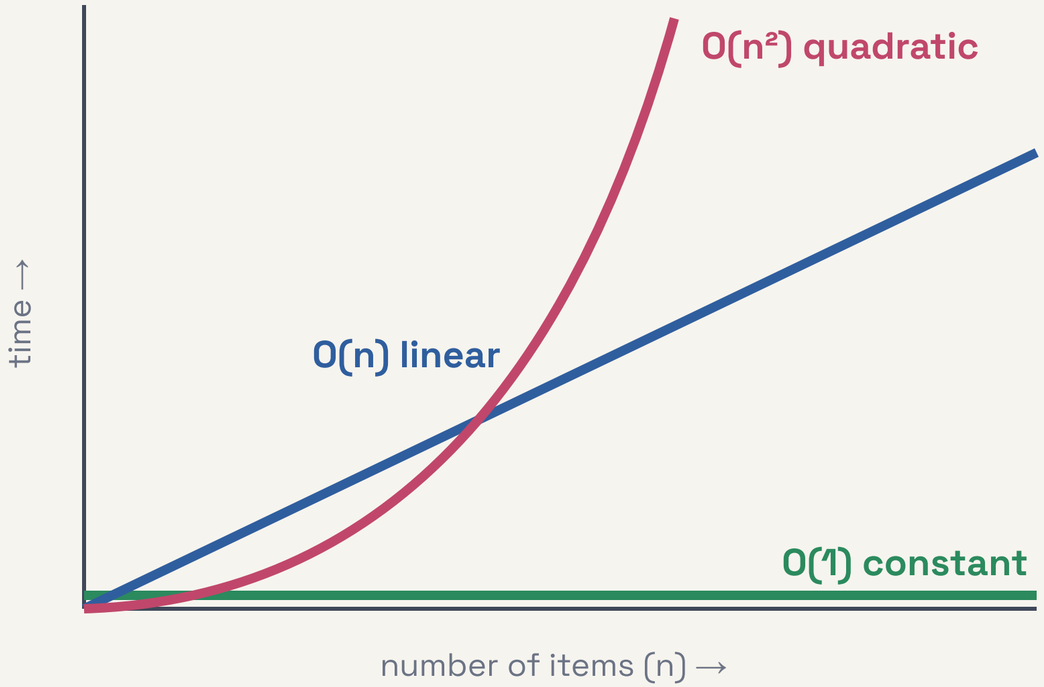
How lists, dicts and sets work inside, and how to pick the right one so your code stays fast.

IN THIS MODULE

Big-O: why it matters	01_big_o.py
Lists & tuples under the hood	02_lists_and_tuples.py
Dicts, sets & hashing	03_dicts_and_sets.py
The collections module	04_collections.py
dataclasses	05_dataclasses.py
Demo: log analyzer	06_demo_log_analyzer.py
Exercise: library tracker	ex09_library_tracker.py

LESSON · BIG-O

How does the work grow when the data grows?



OPERATION	LIST	DICT / SET
`x in …`	O(n)	O(1)
read by index / key	O(1)	O(1)
add at the end	O(1)	O(1)
insert at the front	O(n)	–

MEASURED: "is the last item in here?" × 200

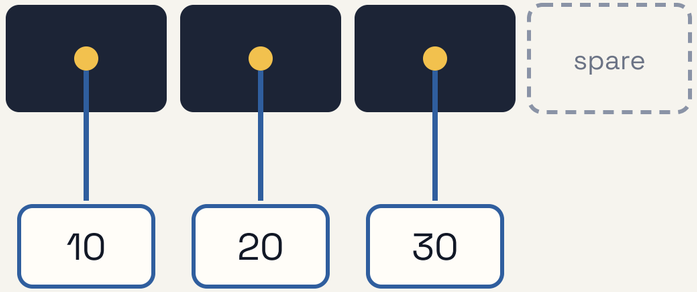
```
    1,000 items | list:    1 ms | set: 0.01 ms
  100,000 items | list:  176 ms | set: 0.02 ms
1,000,000 items | list: 2304 ms | set: 0.01 ms
```

Checking membership a lot? Convert to a **set** once.

LESSON · LISTS & TUPLES UNDER THE HOOD

A list is a row of references, with spare room

a = [10, 20, 30] (capacity 4)



append → spare slot: **fast**

insert(0, x) → shift all: **slow**

each slot = a reference

```
b = a           # same list, 2 names
c = a.copy()    # new outer list (shallow)
d = copy.deepcopy(grid)  # copies inner lists too
```

LIST

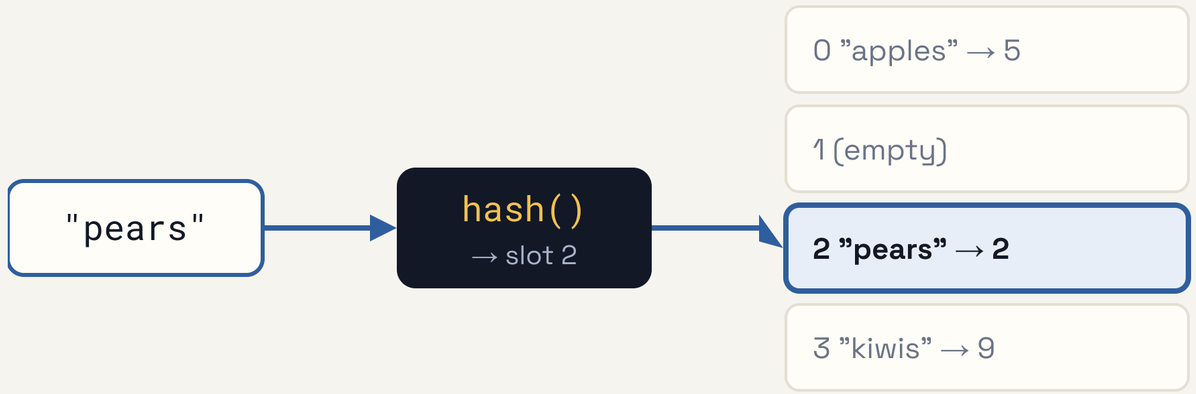
Changeable. Grows and shrinks. Use for **collections of similar items**.

TUPLE

Read-only and lighter (56 vs 72 bytes for 2 items). **Hashable**, so it can be a dict key. Use for **fixed records**: `(x, y)`, RGB, DB rows.

⚠️ A shallow copy shares the inner lists. Nested data? Use **copy.deepcopy**.

Dicts don’t search. They jump straight to the answer.



One calculation, no matter how many keys → **O(1)**

> Keys must be **hashable** = immutable: `str` `int` `tuple` `frozenset`. Not list, dict or set.

SET OP	MEANS
`a | b`	in either (union)
`a & b`	in both (intersection)
`a - b`	only in a (difference)
`a ^ b`	in exactly one

> Dicts keep **insertion order**. Dedupe and keep order: `list(dict.fromkeys(xs))`

LESSON · COLLECTIONS

Four containers that delete boilerplate

COUNTER

Count anything

```
Counter(words).most_common(2)
# [('the', 3), ('and', 2)]
```

DEFAULTDICT

No more "if key not in dict"

```
by_team = defaultdict(list)
by_team[team].append(name)
```

DEQUE

Fast at both ends · keep the last N

```
recent = deque(maxlen=3)
recent.append(page)  # oldest drops off
```

NAMEDTUPLE

A tuple with field names

```
Point = namedtuple("Point", ["x", "y"])
Point(3, 7).x   # 3
```

LESSON · DATACLASSES

Classes for data, without the boilerplate

```python
@dataclass
class Product:
    name: str
    price: float
    qty: int = 0                                # default
    tags: list = field(default_factory=list)

    def total(self) -> float:
        return self.price * self.qty

Product("Laptop", 899.0, 2)
# Product(name='Laptop', price=899.0, qty=2, tags=[])
```

YOU GET FOR FREE

__init__ · **__repr__** · **__eq__**, built from the fields.

OPTIONS

`frozen=True` → read-only and hashable
`order=True` → sortable by the fields
`asdict(obj)` → plain dict (great for JSON)

⚠️ Never `tags: list = []`. Every object would share one list. Use **field(default_factory=list)**.

DEMO · LOG ANALYZER

One question, one right data structure

QUESTION	TOOL
One clean record per log line?	**@dataclass Entry**
Most popular pages?	**Counter** → `most_common(3)`
Which pages did each user visit?	**defaultdict(set)**
How many unique visitors?	**set** comprehension
The last 3 server errors?	**deque(maxlen=3)**

OUTPUT

```
Top pages:
  /home  4 hits
  /cart  3 hits
  /shop  2 hits

Unique visitors: 4

Last 3 server errors:
  10:04 /cart (raj)
  10:07 /cart (vishal)
  10:09 /cart (li)
```

EXERCISE 9 · YOUR TURN

Build a library tracker

The task: fill in 6 TODOs

1	Make `Book` a **dataclass** (isbn, title, author)
2	`build_index` : a **dict** isbn → Book (O(1) lookups)
3	`most_borrowed` : top n with **Counter**
4	`titles_by_author` : **defaultdict(list)**
5	`unique_borrowers` : a **set**
6	`ReturnLog` : the last 3 returns with **deque(maxlen=3)**

```
$ python exercises/
    ex09_library_tracker.py
All checks passed!
```

Starter:
exercises/ex09_library_tracker.py
Solution:
solutions/ex09_library_tracker.py

> Done? Open a **Pull Request** and I'll review your code.